Exploration App — Design Document v3 (Build-Ready)

Status: reviewed and answered by you. Remaining open items are explicitly marked. This is the spec to build Days 1–5 against.

---

1. Summary

A mobile app (Expo / React Native) that turns real-world walking into a fog-of-war exploration game. The map starts desaturated; walking through an area reveals it in full color, permanently. Individual outings are auto-logged as "adventures" in a personal journal, which the user can rename and annotate. Progress is measured against a scope the user picks: City, State/Region, Country, or World.

Core loop: **walk → reveal → capture landmarks → look back on it in the journal.**

---

2. Core Features

2.1 Fog-of-war map

What it does: Renders a real, legible base map (roads, water, parks) that's desaturated everywhere by default. As the user walks, a colored "revealed" region grows around their path and stays revealed permanently.

Use case: User opens the Map tab. They see their city mostly gray, with color patches wherever they've physically walked before. As they walk right now, the area around their live position gradually reveals in color in near-real-time.

How it works:

- `expo-location` continuously watches GPS position while the Map screen (or a background session) is active.
- Each GPS point gets buffered into a small circle (turf.js) and unioned into a running "visited area" polygon.
- That polygon drives a mask on the map rendering: color shows through only inside it, everywhere else stays desaturated.
- Visited-area polygon is persisted (Supabase) per user so it's there on next app open, not recalculated from scratch.

Needed to build:

- `@rnmapbox/maps`, `expo-location`, `turf.js`
- Supabase table storing the user's visited-area geometry (likely as GeoJSON)
- A mask/style layer that responds to that geometry at render time

Resolved: buffer radius (reveal "resolution") will be tuned during real-world testing rather than fixed now — target feel is "not pixel-exact to the user, not so wide it feels unearned."

2.2 Scope selector (City / State / Country / World)

What it does: Lets the user choose the denominator for their "% explored" stat, since a % of your city is meaningful and a % of the world is not.

Use case: User taps the location chip on Home ("San Francisco ▾"). A dropdown shows City, State, Country, and World, each with its own percentage, calculated instantly from data already tracked. They pick one; Home and Map both reflect that scope until changed again.

Scoping logic, confirmed so far:

- **City / State / Country**: area-based — revealed area ÷ that region's boundary area. Requires boundary polygons for whatever administrative region the user is currently in (or has selected).
- **World**: NOT area-based (raw % of Earth's land is meaningless at ~0.0004%). Instead: **% of countries visited**, e.g. "14 of 195 countries." A country only counts as "visited" if the user has at least one qualifying session there (not just a technical GPS ping, e.g. an airport layover) — exact threshold TBD, likely reusing the same session-inclusion filter as adventures (Section 2.3).

Build order (confirmed): City first. State/Region and Country come later; World ("countries visited") can be built roughly alongside Country since it reuses country-level reverse geocoding either way.

Needed to build:

- A boundary-polygon source for city shapes — **not yet finalized.** Mapbox does not include this for free: administrative boundary polygons are a separate paid product, **Mapbox Boundaries** (SLA-based pricing, outside normal Mapbox usage tiers). Cheaper alternatives to evaluate: **Natural Earth** (free, public domain — strong at country/state level, thinner at city level) or **OpenStreetMap boundary extracts** via Nominatim/Overpass API (free, has city-level polygons, more query/setup work). City boundaries are also the least standardized tier in general (city limits vs. metro vs. urban area all differ) — worth a short dedicated decision pass before Day 3, not blocking Day 1–2 work.
- A "countries visited" table/set per user, updated whenever a session's location resolves to a country not already in the set
- Reverse geocoding at the country level (Mapbox Geocoding API) to know which country a session happened in

Still open:

- Final choice of city boundary data source (Natural Earth vs. OSM extract vs. revisit Mapbox Boundaries pricing).
- Default scope on first launch — assumed City, not yet explicitly confirmed.

2.3 Session detection (GPS → discrete "adventures")

What it does: Turns a continuous GPS stream into discrete walking sessions, filtering out noise.

Use case: User goes for a walk. The app silently starts logging once real movement is detected, keeps logging as they walk (including through short pauses), and stops once they've been still for a while. If the walk was too short/trivial, nothing gets saved — no clutter in the journal.

Pipeline (as designed so far):

Stage	Rule
Data integrity	Discard/flag points implying >~15 km/h sustained (likely vehicle, not walking)
Session start	~20–30m movement within ~2 min
Session continues	Keep appending points while movement continues
Session end	~3–5 min of no meaningful movement
Inclusion filter	Only saved if ≥150m cumulative distance AND ≥3 min active movement
Quality signal (not inclusion)	% of session's path that was newly revealed, used for sorting/future gamification

Sessions that fail the inclusion filter are discarded, not stored.

Resolved: background tracking is in scope. Sessions continue logging if the phone is locked or the app is backgrounded. The user-facing expectation is explicitly modeled on Find My / Life360 — i.e., it's understood and accepted that continuous background GPS costs battery, similar to those apps. Product goal is to optimize as much as reasonably possible (e.g. adaptive location accuracy/update frequency, `expo-location`'s background modes and deferred updates) without pretending the cost doesn't exist. This needs an explicit "background location" permission request flow (iOS in particular requires a distinct "Always Allow" prompt, separate from "While Using") and corresponding App Store review disclosure — worth scoping as its own small Day 1–2 task, not an afterthought bolted onto the main location watcher.

Needed to build:

- A background-capable location watcher (`expo-location` background modes, iOS "Always Allow" permission flow, Android background location permission)
- Battery-optimization pass on update frequency/accuracy (exact tuning deferred to real-device testing, similar to Section 2.1's reveal resolution)
- State machine logic (start/continue/end) consuming the same GPS stream that drives the map reveal (Section 2.1) — one subscription, two consumers
- Reverse geocoding call on session end (for the adventure's default title, Section 2.4)

2.4 Recent Adventures / Journal

What it does: Every saved session becomes an "Adventure" — a journal entry with an auto-generated, editable title and an optional note.

Use case: User finishes a walk. Later, they open the Journal tab and see it listed under "August 2026," titled "Mission District" by default. They tap in, see the session's actual stats (read-only) and rename it to "Sunday market walk," adding a note about a bakery they liked.

Confirmed UI (from finalized designs):

- **Journal list**: connected timeline within each month, grouped by month, note-preview snippet shown on cards that have one, small colored dot per entry (color meaning **not yet defined** — flagged in Section 2.4a below).
- **Log detail/edit page**: shows a mini path map, four stats (% explored, landmarks discovered, distance traveled, time elapsed), date/time, and an editable title + note (150-character limit on note, ~30-char limit on title).
- **Only title and note are editable.** No manual entry creation — adventures only come from real sessions.
- Home page surfaces the 2 most recent adventures.

Needed to build:

- Supabase `adventures` table (title, note, path geometry, timestamps, distance, revealed %, landmark count)
- Mapbox reverse geocoding for default titles
- Read/edit UI wired to that table (title + note fields only)

2.4a Resolved during design review

- **Timeline dot color: blue = at least one landmark was discovered during that session; green = default (no landmark discovered).** Applies consistently across the journal timeline.

- **Zero-landmark sessions**: confirmed fine to display as "0 landmarks" — not treated as an error/empty state.
- **Journal header stat**: relabeled to **"12 landmarks discovered"**, counting **unique landmarks** (not total discovery events) across all time.
- **Distance/time precision language**: photo-import and Strava-import are both explicitly future scope (Section 5) — the `is_estimated` schema flag is deferred along with them, not needed for the current build.

2.5 Landmarks

What it does: Physical points of interest that can be "discovered" by visiting and photographing them.

Use case: User is walking and sees a landmark pin appear on the map (undiscovered ones are grayed/unnamed). Once within a set radius, the camera icon at the bottom of the map screen becomes tappable. They take a photo; if it's verified as being taken at the landmark's location, it's marked discovered and added to their stats.

Confirmed UI (from finalized map design):

- Camera icon only becomes active within a set radius of a landmark.
- Nearby-landmarks list lives in an expandable bottom tab on the map: shows distance to each; discovered ones are named and tappable (opens the journal session they were found during); undiscovered ones show as "Undiscovered Landmark."

Verification approach (resolved): metadata-based, not computer vision. A captured photo is verified by checking its EXIF GPS coordinates fall within the landmark's radius (likely also checking the timestamp is recent/during an active session). This does **not** verify the photo actually depicts the landmark — someone standing in the right spot could photograph anything and it would count. Chosen deliberately as the lower-scope v1 approach; explicitly not doing on-device ML or cloud vision.

Dataset (resolved): fixed and hard-coded. Landmarks are not user-submittable and not editable by users. Source of the actual landmark list/coordinates is **not yet chosen** — candidates would include curated data pulled from Mapbox POI data or a manually assembled list, but this needs its own sourcing decision before Day 4–5.

Needed to build:

- `expo-camera` for capture
- EXIF GPS metadata check against landmark radius (+ likely a timestamp/active-session check)
- Landmarks table (lat/lng, name, photo, point value if used) — hard-coded/seeded data, no user-write path

- Per-user discovered-landmarks join table

Still open:

- Landmark dataset source (where the fixed list of coordinates/names actually comes from).

2.5a Location Notes ("field notes") — new feature, resolved this round

What it does: A Dark-Souls-style public messaging feature. Users can drop a short, character-limited note at their exact current location, anywhere on the map — not just at landmarks. Other users walking near that spot can see and read it (not react to it — reading it *is* the interaction).

Use case: A user finds a great viewpoint and leaves a note like "great sunset spot, come back at golden hour." Another user, unrelated to the first, is walking nearby days later. Once they cross into the note's visibility radius, it appears as a marker on their map; if they get closer still (inside a tighter radius), the app sends a push notification alerting them a note is nearby. They tap it to read it. There's no reply, reaction, or authorship visible beyond the note text itself — reading it on the map is the whole interaction.

Confirmed mechanics:

- Always available — not gated to landmarks, can be left at any location.
- Short, character-limited text (same general pattern as the adventure note field).
- **Visibility radius:** placeholder default **300–500m** (your original number was 3km — flagged as likely too large for a walking-scale feature; needs real-world tuning either way).
- **Notification radius:** placeholder default **75–150m** (your original number was 1km — same flag as above).
- A user's own written notes are visible to themself via a toggleable **"view notes"** button on the map — **not yet designed**, needs a UI pass.
- Users cannot see other users' notes until within the visibility radius.
- No reactions — clicking a nearby note simply opens it to read.
- Push notification on proximity, **with a settings toggle to disable it.**
- **Content moderation (filtering inappropriate notes) is explicitly future scope**, not part of the current build. No moderation tooling, reporting, or filtering exists in v1 — worth being deliberate about this as a known gap given notes are public-facing UGC.

Needed to build:

- `expo-notifications` for push notifications — new to the stack, not previously scoped
- A location-triggered background check (proximity to nearby notes), likely riding along the same background location infrastructure as Section 2.3, but logically a

separate system
- A settings toggle for note-proximity notifications (this means Settings needs at least one real, functional feature now, not just a placeholder screen — see Section 2.7)
- A `notes` table: author (device ID), lat/lng, text, timestamp — no reaction/reply fields needed
- A "view my notes" UI on the map (not yet designed)
- Two radius values (visibility, notification) — placeholder numbers above, tune during testing

Still open:

- Final radius values (placeholder 300–500m visibility / 75–150m notification pending real-world testing).
- "View notes" toggle UI design.
- Moderation/reporting system — deferred to future scope per Section 3, but worth revisiting before any real public launch given it's unmoderated UGC visible to strangers.

2.6 Map navigation & session indicator

What it does: The map screen has no bottom nav (by design, for a clean/immersive feel); a floating back button provides the way home. A pulsing-dot pill shows an active session is being logged.

Confirmed UI (from finalized map design):

- Back button: top-left, circular, floating.
- Session indicator: pill showing "● Session active · 12 min," positioned near the scope/location chip.
- Scope/% chip: top-right, tap opens the City/State/Country/World dropdown (Section 2.2).
- Bottom-right: two floating circular buttons — notepad (location note) and camera (landmark capture, conditionally enabled).
- Bottom-center: expandable tab for nearby landmarks list.

Needed to build: Straightforward UI work once the underlying session/landmark logic above exists — no new data model beyond what's already listed.

2.7 Profile

What it does: Shows the user's name, all-time stats, a friends list, and access to settings.

Use case: User taps Profile, sees their name/avatar, total adventures/landmarks/city %, a short list of friends, and a settings entry point.

Status: Layout direction chosen (centered header style), but **explicitly deferred in scope** — you said this is intentionally light for now. Friend-profile viewing and settings screen contents are not yet designed.

Needed to build (for the current, minimal version):

- Supabase `users` table (name, avatar, created date)
- Aggregate stat queries (reuses adventure/landmark data already being tracked)
- A `friends` relationship table (even a minimal one) — structure not yet designed (see Section 4)
- A settings screen — contents not yet defined at all

---

3. Explicitly Deferred (you flagged these as later scope)

- Camera capture flow detail (UI polish beyond "camera enables within radius")
- Friends' profile viewing
- Settings screen contents (beyond the one note-notification toggle now required by Section 2.5a)
- Note moderation / inappropriate-content filtering (Section 2.5a) — explicitly future scope, called out again here since it's a real gap in the current plan
- Imported-session data (photo/Strava import) — future scope, Section 5

No longer deferred, now in scope: the notes popup itself (Section 2.5a) — originally listed as later scope, now designed and confirmed in.

---

4. Features Discussed But Not Yet Designed At All

- **Friends system mechanics**: confirmed later scope. Likely entry point will be a search feature (find users by username/handle), but no further detail decided — how friends are added, what's visible on a friend's profile, any social/comparison features are all undesigned.
- **Points system**: mentioned early on as a stretch feature tied to landmark capture; not explicitly re-confirmed this round. Treating as not blocking Day 1–5 either way, but flagging it's still technically undecided rather than formally deferred.

---

5. Future / v2 Features (researched, not being built now)

5.1 Photo metadata (EXIF) import

Backfills exploration data from a user's existing photo library for trips taken before they had the app. Reuses the existing session-clustering and turf.js buffer/union logic, with wider clustering thresholds (time gap ~4–6 hrs, distance jump ~15–20km) since photo cadence is sparse. Distance/time stats from imported sessions should be labeled as

estimates. Reveals buffered circles around individual photo points rather than a connected path, since no path data exists.

5.2 Strava import

Stronger data source than photos (real recorded GPS paths, not sparse points). Real, documented OAuth API exists. Caveats: as of June 2026, Strava's Standard developer tier requires a Strava subscription for continued API access (a recent change); rate limits (200 req/15min, 2000/day) are generous enough for early use; API policy explicitly restricts using Strava data in AI processing pipelines (Section 5.3 of their policy) — worth re-reading closely if this is ever built, since it's a hard contractual restriction, not a style note.

5.3 Rejected sources (evaluated, not viable)

- **Life360**: no official API; only an unofficial, reverse-engineered one that Life360 has actively worked to block.
- **Apple Find My**: no public API of any kind for location history.
- **Google Timeline**: as of the 2024 migration to on-device, encrypted storage, only ~90 days of data is typically available, and export requires a manual, technical, on-device flow — not worth building against.

---

6. Tech Stack (confirmed so far)

- **Framework**: Expo / React Native
- **Map**: Mapbox (`@rnmapbox/maps`), including Geocoding API for reverse geocoding
- **Geometry**: turf.js (buffering, unioning visited-area polygons)
- **Location**: `expo-location`, including background location modes (Section 2.3)
- **Camera**: `expo-camera`, with EXIF metadata read for landmark verification (Section 2.5)
- **Notifications**: `expo-notifications`, for note-proximity alerts (Section 2.5a) — new addition this round
- **Backend/DB**: Supabase, anonymous device-ID-based users (no full auth system yet)
- **Boundary data**: not yet finalized — Natural Earth or OSM extracts under consideration for city-level polygons (Mapbox Boundaries is a paid add-on, not included by default)
- **Dev tooling**: Claude Code, EAS Build for installable dev builds

---

7. Still Open (down to a short list now)

1. **City boundary data source** — Natural Earth vs. OSM extract vs. revisit Mapbox Boundaries pricing (Section 2.2).
2. **Landmark dataset source** — where the fixed, hard-coded list of landmarks actually comes from (Section 2.5).

3. **Field note radius values** — placeholder 300–500m visibility / 75–150m notification, pending real-world testing (Section 2.5a).
4. **"View my notes" toggle UI** — not yet designed (Section 2.5a).
5. **Points system** — technically undecided, not blocking (Section 4).

None of these block starting Day 1–2 build work (map reveal, GPS session detection, background tracking setup). Items 1–2 need answers before their respective features (scope % and landmark seeding) can be finished; 3–4 are UI/tuning passes that can happen in parallel with core build; 5 can wait indefinitely.

Suggested next step: translate Sections 2.1–2.3 (map reveal, City scope, session detection incl. background tracking) into an actual Day 1–5 implementation plan with Claude Code, since those three are the foundation everything else — journal, landmarks, notes — sits on top of.